More Machine Learning

Machine Learning course notes Sources: standard material, listed under References

What this file is really about:

$$\text{methods worth knowing} \rightarrow \text{that the course lectures leave out}$$

How to use this file. Each section stands alone, and has an interactive section to try it. Part C is practice with answers.

Step-up path

You need. The core lectures of this course.

1. Handle missing data and rare classes.
2. Reason about how much data suffices.
3. Separate, factorise and decode hidden structure.
4. Learn online and from unlabelled data.
5. Test hypotheses and model survival.

You can stop when you can recognise when one of these tools fits your problem.

Contents

Part A Maths From the Ground Up

A1 Linear discriminant analysis

Model each class as a Gaussian with a **shared** covariance. The log-odds are then linear in x, with weight direction $w = \Sigma^{-1}(\mu_1 - \mu_0)$. With a separate covariance per class the boundary becomes quadratic, which is **QDA**.

A2 Splines and additive models

A **spline** joins polynomial pieces at knots with smooth joins. A **generalised additive model** fits one smooth curve per feature and adds them, staying interpretable while allowing curvature.

A3 Imbalanced classes

Rare positives are easy to ignore. Options are class weights, resampling, or **SMOTE**, which interpolates between a minority point and one of its minority neighbours. Change the threshold too, and evaluate with precision and recall rather than accuracy.

A4 Missing data

Data can be missing completely at random, at random given other features, or not at random. Dropping rows wastes data and can bias results. Impute using statistics from training data only, and add an indicator column when missingness may carry information.

A5 Encoding categories

One-hot encoding grows with the number of categories. **Target encoding** replaces a category with the mean label, which leaks unless computed out of fold. **Feature hashing** caps the dimension.

A6 Learning theory

VC dimension is the size of the largest set a model family can label in every possible way. A linear classifier in d dimensions has VC dimension $d + 1$. A finite class of $|H|$ hypotheses needs about

$$n \geq \frac{1}{\varepsilon}\left(\ln |H| + \ln \frac{1}{\delta}\right)$$

examples for a zero-training-error hypothesis to have true error below ε with probability at least $1 - \delta$.

A7 Independent component analysis

PCA finds uncorrelated directions. **ICA** finds statistically independent ones, by looking for the most non-Gaussian projections. It can separate mixed signals, as with two voices recorded by two microphones.

A8 Non-negative matrix factorisation

Factor $A \approx W H$ with all entries non-negative. Components can only add, so they often look like parts, such as topics in a word-count table.

A9 Hidden Markov models

A hidden state follows a Markov chain and emits observations. The **Viterbi** algorithm finds the most likely state sequence by dynamic programming:

$$\delta_t(s) = \max_{s'} \delta_{t-1}(s')\, a_{s's}\, b_s(o_t).$$

A10 Online learning

Data arrive one at a time. **Regret** compares your total loss with that of the best fixed choice in hindsight. The **Hedge** algorithm keeps a weight per expert and multiplies it by $e^{-\eta \ell}$ after each loss.

A11 Semi-supervised learning

With few labels and many unlabelled examples, **self-training** predicts labels for the most confident unlabelled points, and retrains. Wrong confident predictions can reinforce themselves, so a confidence threshold matters.

A12 Association rules

For a rule $A \Rightarrow B$:

$$\text{support} = P(A \cap B), \quad \text{confidence} = P(B \mid A), \quad \text{lift} = \frac{P(B \mid A)}{P(B)}.$$

A lift above 1 means A and B occur together more often than chance.

A13 Hypothesis tests

A **p-value** is the probability, if the null hypothesis were true, of data at least this extreme. The two-sample **t-test** compares means, and the **chi-square test** compares observed and expected counts:

$$\chi^2 = \sum \frac{(O - E)^2}{E}.$$

Careful. A p-value is not the probability that the null hypothesis is true.

A14 Survival analysis

When some subjects have not yet had the event, their data are **censored**. The **Kaplan-Meier** estimator multiplies the survival fraction at each event time:

$$S(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right).$$

A15 Multi-label problems

An example can have several labels at once. Train one output per label, using a sigmoid and not a softmax. **Hamming loss** is the fraction of label slots predicted wrongly.

A16 Linear discriminant analysis, derived

With class densities $\mathcal{N}(\mu_k, \Sigma)$ sharing one covariance, the log odds are

$$\log \frac{P(1 \mid x)}{P(0 \mid x)} = \log \frac{\pi_1}{\pi_0} - \tfrac{1}{2}(\mu_1 + \mu_0)^\top \Sigma^{-1}(\mu_1 - \mu_0) + x^\top \Sigma^{-1}(\mu_1 - \mu_0).$$

The quadratic terms $x^\top \Sigma^{-1} x$ cancel because the covariance is shared, so the boundary is linear. With different covariances they remain, giving **QDA**.

A17 A generalisation bound from Hoeffding

For a single hypothesis, with loss in $[0, 1]$ and n examples,

$$P\big(|R - \hat{R}| > \epsilon\big) \le 2e^{-2n\epsilon^2}.$$

Apply a union bound over a finite class of $|H|$ hypotheses and solve for ϵ. With probability at least $1 - \delta$, every h satisfies

$$R(h) \le \hat{R}(h) + \sqrt{\frac{\log |H| + \log(2/\delta)}{2n}}.$$

Larger classes need more data, but only logarithmically. Infinite classes use the **VC dimension** in place of $\log |H|$.

A18 Kaplan-Meier, worked

Survival drops at each event by the fraction still at risk that fail:

$$\hat{S}(t) = \prod_{t_i \le t} \Big(1 - \frac{d_i}{n_i}\Big).$$

Take five subjects with event times 2, 3, 5 and censoring at 3 and 6. At $t = 2$: $\hat{S} = 4/5$. At $t = 3$: one event among 4 at risk, giving $0.8 \times 3/4 = 0.6$. The censored subject then leaves. At $t = 5$: one event among 2 at risk, giving $0.6 \times 1/2 = 0.3$.

A19 Forward-backward and Baum-Welch

$\alpha_t(i) = P(x_{1:t}, s_t = i)$ is computed forward, and $\beta_t(i) = P(x_{t+1:T} \mid s_t = i)$ backward. Then the state posterior is $\gamma_t(i) \propto \alpha_t(i)\beta_t(i)$. **Baum-Welch** is EM: compute these, then re-estimate transitions and emissions as expected counts.

Part B In Code

B1 Class weights

```
LogisticRegression(class_weight='balanced').fit(X, y)
```

B2 Imputing inside a pipeline

```
make_pipeline(SimpleImputer(strategy='median'), StandardScaler(), model).fit(X_train, y_tra
```

B3 LDA from scratch

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

rng = np.random.default_rng(0)
X = np.r_[rng.normal([0, 0], 1, (300, 2)), rng.normal([2, 1], 1, (300, 2))]
y = np.r_[np.zeros(300), np.ones(300)]

m0, m1 = X[y == 0].mean(0), X[y == 1].mean(0)
S = (np.cov(X[y == 0].T) + np.cov(X[y == 1].T)) / 2
w = np.linalg.solve(S, m1 - m0)
b = -0.5 * (m0 + m1) @ w
acc = ((X @ w + b > 0) == y).mean()
sk = LinearDiscriminantAnalysis().fit(X, y).score(X, y)
print(round(acc, 3), round(sk, 3))
assert abs(acc - sk) < 0.01
```

B4 Kaplan-Meier

```
import numpy as np

def kaplan_meier(time, event):
    order = np.argsort(time); time, event = time[order], event[order]
    s, out = 1.0, []
    for t in np.unique(time[event == 1]):
        at_risk = (time >= t).sum()
        deaths = ((time == t) & (event == 1)).sum()
        s *= 1 - deaths / at_risk
        out.append((float(t), round(float(s), 3)))
    return out

km = kaplan_meier(np.array([2, 3, 3, 5, 6]), np.array([1, 1, 0, 1, 0]))
print(km)
assert km == [(2.0, 0.8), (3.0, 0.6), (5.0, 0.3)]
```

B5 Viterbi

```
import numpy as np

def viterbi(pi, A, B, obs):
    d = np.log(pi) + np.log(B[:, obs[0]])
    back = []
    for o in obs[1:]:
        s = d[:, None] + np.log(A)
        back.append(s.argmax(0))
        d = s.max(0) + np.log(B[:, o])
    path = [int(d.argmax())]
    for b in reversed(back):
        path.append(int(b[path[-1]]))
    return path[::-1]

pi = np.array([0.6, 0.4]); A = np.array([[0.7, 0.3], [0.4, 0.6]])
B = np.array([[0.1, 0.4, 0.5], [0.6, 0.3, 0.1]])
print(viterbi(pi, A, B, [0, 1, 2]))
assert viterbi(pi, A, B, [0, 1, 2]) == [1, 0, 0]
```

B6 SMOTE from scratch

```
import numpy as np

def smote(X, n, k=5, seed=0):
    rng = np.random.default_rng(seed)
    d = ((X[:, None] - X[None]) ** 2).sum(-1)
    near = d.argsort(1)[:, 1:k + 1]
    i = rng.integers(len(X), size=n)
    j = near[i, rng.integers(k, size=n)]
    return X[i] + rng.random((n, 1)) * (X[j] - X[i])     # a point on the segment to a neig

X = np.random.default_rng(1).normal(size=(30, 2))
new = smote(X, 100)
print(new.shape)
assert new.shape == (100, 2)
```

Part C Practice

P1. What is the VC dimension of a linear classifier in 5 dimensions?

6.

P2. $P(A \cap B) = 0.1$, $P(A) = 0.2$, $P(B) = 0.25$. Find confidence and lift.

Confidence 0.5, and lift 2.

P3. Observed counts 30 and 10, with 20 expected in each. Find the chi-square statistic.

$5 + 5 = 10$.

P4. 10 at risk and 2 events, then 6 at risk and 3 events. Find the Kaplan-Meier survival.

$0.8 \times 0.5 = 0.4$.

P5. Should a multi-label model end in softmax?

No. Labels are not exclusive, so each needs its own sigmoid.

P6. With $|H| = 1000$, $n = 5000$, $\delta = 0.05$, find the bound on the gap.

$\sqrt{(6.908 + 3.689)/10000} = 0.0326$.

P7. Why is the LDA boundary linear?

The shared covariance makes the quadratic terms cancel in the log odds.